

이 문서는 가이드라인 일뿐 프로젝트의 성질에 따라 변경하거나 선택하여
수행

생성형 AI Web App 기획 및 개발 프로젝트

각 단계별 주요내용 및 세부내용

1. 문제정의 및 AI Web App 서비스 기획

Design Thinking 기반 개발 계획 수립

문제 정의 및 아이디어 발굴

- 산업·업무 현장의 문제 및 사용자 Pain Point 도출
- AI 적용 가능 문제 선정

Design Thinking 프로세스 적용

Empathize(공감) → Define(문제정의) → Ideate(아이디어 발상)

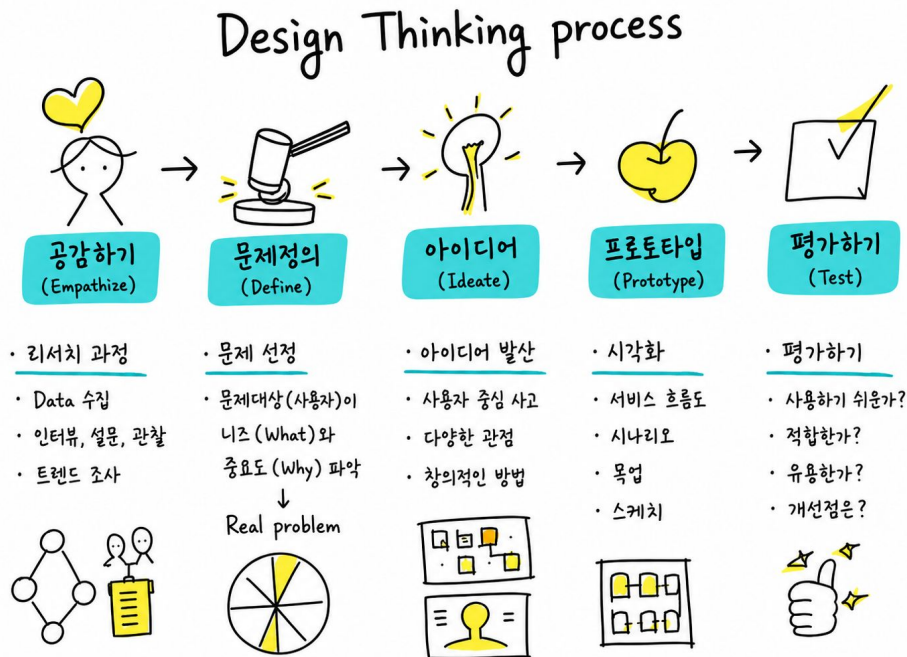
→ Prototype(모형제작) → Test(테스트:피드백)

프로젝트 개발 계획 수립

- 개발 범위, 일정, 역할 및 산출물 정의
- MVP 개발 범위 설정

활용 도구

- Notion, Figma, Slack
- Jira, Trello, GitHub Project
- Google Workspace, Microsoft 365



3. 서비스 컨셉 및 MVP 정의

서비스 아이디어 구체화

- AI 적용 서비스 컨셉 정의
- 핵심 문제 및 해결방안 정의

서비스 가치 제안

- Value Proposition(가치제안) 정의 고객이 왜 다른 곳이 아니라 우리 제품이나 서비스를 사용해야 하는지를 명확하게 보여주기
- 타 서비스와의 차별화 요소 도출

PoC(개념검증), MVP 설계

- 핵심 기능 선정
- 우선순위 및 개발 범위 설정
- '기술적/실행적 가능성'을 확인 UI/UX 구현에 시간을 쓰지 않고 API 연동과 프롬프트 검증에 집중
화면은 Streamlit이나 Gradio 같은 도구를 활용해 텍스트 입력창과 결과창만 구현.
성공 지표(KPI) 설정 : LLM API가 우리 도메인 데이터를 기반으로 5초 이내에 정확한 답변을 생성하는가?

활용 도구

- Notion, Figma
- Jira / Trello
- Excel / Google Sheets

4. UX/UI 학습 및 서비스 화면 설계

UX/UI 기본 개념

- 사용자 경험 및 인터페이스 설계
- 사용성 및 접근성 고려

화면 및 사용자 흐름 설계

- Sitemap
- User Flow
- Storyboard, Wireframe

Prototype 제작

- 주요 화면 설계
- 사용자 인터랙션 설계

활용 도구 및 기술

- Figma
- Canva, 미리 Canvas
- Adobe XD
- HTML / CSS / JavaScript

5. Web App 시스템 아키텍처 설계

서비스 전체 구조 설계

- Front-end / Back-end / AI Server 구조
- DB / ML Model / Vector DB / LLM API 구성

데이터 및 요청 흐름 설계

- 사용자 요청 → API → AI 처리 → 결과 반환

서버 및 API 구조 설계

- `/`, `/api/*`, `/ai/*` 등 서비스 경로 설계

아키텍처 문서화

- 시스템 구성도
- 데이터 흐름도
- API 연동 구조도

활용 도구

- Figma
- Canva
- AWS Architecture Diagram

6. Web App 개발 환경 및 협업 환경 준비

개발환경 구성

- Python 개발환경
- Node.js / npm 환경
- VS Code 기반 개발환경

프로젝트 구조 설정

- Front-end / Back-end / AI Server 디렉터리 구성

버전 및 패키지 관리

- Python `venv`
- `pip`
- npm

협업환경 구축

- Git Repository, GitHub
- Branch / Commit / Pull Request 전략

활용 도구

- VS Code
- Git / GitHub
- Docker
- Postman

7. React 기반 Front-end 개발

Web Front-end 기본

- HTML / CSS / JavaScript

React 개발

- React Component
- Props / State
- Event 처리

React 기반 SPA 구현

- 페이지 및 컴포넌트 구성
- 사용자 입력 및 결과 화면 구현

Back-end API 연동

- HTTP Request
- JSON 데이터 처리

활용 기술/도구

- React (Vite)
- JavaScript / TypeScript
- Axios / Fetch
- VS Code

8. FastAPI 기반 Back-end 및 REST API 개발

Web API 개념

- HTTP
- REST API
- GET / POST / PUT / DELETE

FastAPI 개발

- API Endpoint 설계
- Request / Response 처리
- Pydantic 데이터 검증

비즈니스 로직 구현

- 회원/서비스/게시물 등의 CRUD
- Front-end와 데이터 연동

API 문서화 및 테스트

- FastAPI 자동 문서화 / Swagger UI

활용 기술/도구

- Python
- FastAPI / Uvicorn
- Swagger / OpenAPI
- Postman

9. 데이터베이스 설계 및 구현

데이터베이스 설계

- 데이터 항목 및 관계 정의
- ERD 설계

MySQL 구현

- Database / Table 생성
- Primary Key / Foreign Key

SQL 활용

- SELECT / INSERT / UPDATE / DELETE
- JOIN / GROUP BY 등

Web App 연동

- FastAPI ↔ MySQL 연동
- ORM 활용(선택)

활용 기술/도구

- SQL
- MySQL
- SQLAlchemy
- MySQL Workbench

10. 데이터 수집·분석 및 전처리

데이터 수집

- CSV / Excel / Open Data
- API 기반 데이터 수집

데이터 탐색 및 분석

- EDA
- 통계적 특성 분석

데이터 전처리

- 결측치 / 이상치 처리
- 중복 제거
- 데이터 변환 및 정규화

학습 데이터셋 구축

- Train / Validation / Test 데이터 구성

활용 기술/도구

- Python
- Pandas , NumPy
- Matplotlib
- Jupyter Notebook
- Google Colab

11. 인공지능 모델링

AI 문제 유형 정의

- 분류 / 회귀 / 군집화 등
- 예측 문제 정의

AI 모델 선정

- 문제와 데이터 특성에 따른 모델 선택

모델 학습

- 학습 데이터 구성
- Feature Engineering
- Model Training

모델 평가

- Accuracy / Precision / Recall / F1
- MAE / MSE / RMSE 등

활용 기술/도구

- Python
- Scikit-learn
- XGBoost / LightGBM
- PyTorch 또는 TensorFlow
- Jupyter / Google Colab

12. AI 모델 서비스화

학습 모델 저장

- `.joblib`, `.pkl` 등의 모델 저장

Prediction 기능 구현

- 입력 데이터 전처리
- 모델 추론

AI Server 구축

- FastAPI 기반 Prediction API

Web App 연동

- Front-end → API → AI Model → 결과 반환

활용 기술/도구

- Python
- Scikit-learn / PyTorch
- Joblib / Pickle
- FastAPI
- Uvicorn
- Postman

13. Prompt Engineering 및 생성형 AI 활용

LLM 기본 이해

- Token
- Context
- Temperature
- 모델별 특성

Prompt Engineering 설계

- Role / Context / Task / Output Format
- Zero-shot / Few-shot Prompt

구조화된 응답 설계

- JSON 출력
- Structured Output

Prompt 평가 및 개선

- 정확성 / 일관성 / 유용성 평가
- 반복적인 Prompt Optimization

활용 도구

- ChatGPT / Gemini / Claude 등 LLM
- OpenAI API / Gemini API
- Prompt 관리용 Notion

14. OpenAPI 기반 LLM 연동

LLM API 연동

- OpenAI API
- Google Gemini API
- Hugging Face API 등

생성형 AI 기능 구현

- 텍스트 생성
- 요약
- 분류
- 질의응답

API 비용 및 사용량 관리

- Token 사용량 확인
- API 호출 최적화

활용 기술/도구

- Python
- REST API
- OpenAI API
- Gemini API
- Hugging Face
- Postman

15. RAG 기반 생성형 AI 서비스 구현

RAG 개념 이해

- LLM과 외부 지식 결합, RAG Architecture 이해

데이터 및 문서 처리

- PDF / TXT / CSV / Web 데이터 수집, 문서 Chunking

Embedding

- 문서 Vectorization, Embedding Model 활용

Vector DB 구축

- Vector 저장, Similarity Search

LLM 연계

- 검색 결과 → Prompt → LLM → 응답

활용 기술/도구

- Python
- LangChain / LlamaIndex
- ChromaDB / FAISS / Qdrant 등 Vector Database
- OpenAI Embedding
- Hugging Face

16. Tools / Function Calling 기반 AI 기능 구현

AI Agent 및 Tool 개념 이해

- LLM이 외부 기능을 호출하는 구조

Function Calling 구현

- 함수 정의
- Parameter Schema 설계
- 함수 실행 결과 전달

외부 서비스/API 연동

- 검색 API, 데이터 조회 API, 사내 업무 API 등

AI 기반 업무 자동화

- 사용자 질문 분석, 적절한 Tool 선택, 결과 생성

활용 기술/도구

- Python
- OpenAI Function Calling / Tools
- Gemini Function Calling / Tools
- LangChain
- FastAPI
- REST API

17. Prototype 제작 및 사용자 피드백 기반 보완

Prototype 구현

- 핵심 기능 중심 MVP 제작
- 실제 사용자 시나리오 구현

사용자 테스트

- 사용성 테스트
- Task 수행 테스트

피드백 수집

- 사용자 인터뷰
- 설문조사
- 사용 로그 분석

반복적 개선

- 문제점 도출 → 수정 → 재검증

활용 도구

- Figma
- React
- Google Forms
- Notion
- Jira / GitHub Issues

18. Fine-Tuning 및 생성형 AI 모델 고도화

Fine-Tuning 개념 이해

- Prompt Engineering / RAG / Fine-Tuning의 차이 / Fine-Tuning 적용 필요성 판단

학습 데이터 준비

- Domain Dataset 구축 / 데이터 정제 및 Labeling

모델 Fine-Tuning

- 사전학습 모델 활용, LoRA / PEFT 등 경량 Fine-Tuning

모델 평가

- Fine-Tuning 전/후 성능 비교

서비스 적용

- Fine-Tuning 모델 저장, API 또는 AI Server 연동

활용 기술/도구

- Python, PyTorch
- Hugging Face Transformers
- PEFT / LoRA
- Google Colab / GPU
- Hugging Face Hub

19. AI Web App 통합 구현

전체 시스템 통합

- React + FastAPI + AI Server
- MySQL + ML Model + Vector DB
- LLM API 통합

서비스 데이터 흐름 구현

- 사용자 입력 , Backend 처리, AI 처리, 결과 반환

API 통합

- Front-end ↔ Backend
- Backend ↔ AI Server
- AI Server ↔ LLM / RAG

통합 오류 처리

- API 오류, AI 오류, DB 오류 대응

활용 기술/도구

- React
- Python / FastAPI / MySQL
- Vector DB
- LLM API
- GitHub

20. AWS EC2 기반 클라우드 배포

Cloud Computing 기본

- IaaS / Server / Network 이해

AWS EC2 구성

- Instance 생성 / Security Group 설정 / SSH 접속

Web App 배포

- React Build / FastAPI Server / AI Server

서비스 운영

- Process 관리 / Log 확인 / 서버 상태 확인

환경변수 관리

- API Key / DB 접속정보 / `.env` 활용

활용 기술/도구

- AWS EC2 / Ubuntu Linux / SSH
- Nginx
- Git / GitHub
- Docker(선택)

21. Nginx Reverse Proxy 및 서비스 구성

Reverse Proxy 개념

- Client → Nginx → 각 Application Server 구조

서비스 Routing

- `/` → React
- `/api/*` → Backend
- `/ai/*` → AI Server

정적 파일 서비스

- React Build 파일 배포

Web Server 설정

- Nginx Configuration
- Port 및 서버 연결

활용 기술/도구

- Nginx
- Linux / Ubuntu
- SSH
- Vim / Nano
- curl

22. Web App 및 AI 서비스 테스트

Front-end 테스트

- 화면 및 사용자 입력 테스트

API 테스트

- Request / Response 검증
- API Endpoint 테스트

AI 모델 테스트

- Prediction 결과 검증
- 모델 성능 측정

RAG 테스트

- 검색 정확도 / 검색 결과 적합성

LLM 테스트

- 응답 정확성 / Hallucination / 응답 일관성

활용 도구

- Postman / Swagger UI
- Jupyter / Browser DevTools

23. 서비스 고도화 및 비즈니스 모델 수립

사용자 피드백 분석

- 사용자 요구사항 재분석
- 개선 우선순위 설정

서비스 고도화

- AI 기능 추가
- RAG / Agent / Multimodal 기능 확장

비즈니스 모델 설계

- 고객 정의, 가치 제안, 수익 모델, 비용 구조

서비스 차별화

- 경쟁 서비스 분석
- AI 적용에 따른 경쟁력 도출

활용 도구

- Business Model Canvas
- Miro
- Notion
- Google Sheets
- Figma

24. 생성형 AI 보안·윤리·저작권 적용

AI 윤리

- 생성형 AI의 책임 있는 활용
- 편향 및 차별 문제

개인정보 보호

- 개인정보 수집 및 처리, 민감정보 보호

AI 보안

- Prompt Injection
- 악의적 입력
- API Key 노출 방지

저작권

- 학습 데이터 및 생성 결과물의 저작권 검토, 오픈소스 라이선스 확인

활용 기술/도구

- `.env`
- GitHub Secret
- AWS IAM
- GitHub Secret Scanning
- OWASP 기준 참고

25. 프로젝트 결과물 및 포트폴리오 제작

프로젝트 산출물 정리

- 서비스 기획서, 요구사항 정의서, 시스템 구성도, ERD, API 명세서, 프로젝트 수행일지

개발 결과물 정리

- Source Code, GitHub Repository, 배포 URL, Demo Video

AI 관련 결과물

- 데이터 분석 결과, AI 모델 평가 결과
- Prompt / RAG / Fine-Tuning 결과

개인 포트폴리오 구성

- 프로젝트 역할, 문제 해결 과정, 기술 스택, 성과 및 개선점

활용 도구

- GitHub
- Notion
- Figma
- Google Drive
- PowerPoint
- Canva

26. 제안 기획, PT 자료 작성 및 제안 PT

프로젝트 제안 기획

- 문제 및 시장 분석, 서비스 필요성, AI 활용 방안, 기대효과

제안서 작성

- 서비스 개요, 시스템 구성, AI 모델 및 기술 구성, 개발 및 운영 계획

PT 자료 제작

- 발표 Storyline 구성, 핵심 메시지 및 시각화, Demo 시나리오 구성

제안 PT 및 피드백

- 발표 및 시연, 질의응답, 피드백 반영

활용 도구

- PowerPoint
- Canva
- Figma
- Gamma
- Google Slides
- Zoom / Teams

전체 프로젝트 기술 스택

영역	주요 기술/도구	영역	주요 기술/도구
기획/협업	Notion, Miro, FigJam, Jira, GitHub	RAG	Embedding, LangChain/LlamaIndex, Vector DB
UI/UX	Figma, FigJam	Vector DB	ChromaDB, FAISS, Qdrant 등
Front-end	HTML, CSS, JavaScript, React, Vite	Fine-Tuning	Hugging Face, Transformers, PEFT, LoRA, PyTorch
Back-end	Python, FastAPI, Uvicorn	AI Tools/Agent	Function Calling, Tools, LangChain
API	REST API, OpenAPI, Swagger, Postman	Server	Linux/Ubuntu, Nginx
Database	MySQL, SQL, SQLAlchemy, DBeaver	Cloud	AWS EC2, SSH, CloudWatch
Data Analysis	Python, Pandas, NumPy, Matplotlib	형상관리	Git, GitHub, GitHub Issues/Projects
AI/ML	Scikit-learn, XGBoost, PyTorch/TensorFlow	테스트	Postman, Swagger, pytest, DevTools
AI Model Serving	FastAPI, Joblib/Pickle	배포	AWS EC2, Nginx, Docker(선택)
Generative AI	LLM, Prompt Engineering, OpenAI API, Gemini API	문서/발표	PowerPoint, Canva, Figma, Notion